

Linear Models & Analysis

CIS 5450, Fall 2026

Ryan Marcus



Penn
Engineering
UNIVERSITY of PENNSYLVANIA

Outline

- **Course logistics**
- Brief review
- Linear models: interpretation
- Linear models: analysis

Upcoming Deadlines

- HW2 due today at 10pm
 - HW3 released today, due in 3 weeks
- Midterm 1 on Oct 7th (7 days)

Outline

- Course logistics
- **Brief review**
- Linear models: interpretation
- Linear models: analysis

✓
0s

```
!wget -nc https://rmarcus.info/data/yawns.  
df = pd.read_csv("yawns.csv")  
df
```

↔ File 'yawns.csv' already there; not retrieving.

	person_id	group	measure
0	0	CONTROL	no yawn
1	1	CONTROL	no yawn
2	2	CONTROL	no yawn
3	3	CONTROL	no yawn
4	4	CONTROL	no yawn
5	5	CONTROL	no yawn
6	6	CONTROL	no yawn
7	7	CONTROL	no yawn
8	8	CONTROL	no yawn

CONTROL or TREAT

random people into
go yawn in one room,
ers.

Effect: "no yawn" or
"yawn"

Is Yawning Contagious?

- 25% of the control group yawned
- 29% of the treatment group yawned
- Is it a real effect, or just random chance?

✓
0s

```
[9] control_yawners = 4 / (12 + 4)
    treat_yawners = 10 / (24 + 10)

    control_yawners, treat_yawners
```

⇒ (0.25, 0.29411764705882354)

Example adapted from [Dr. Maria Tackett](#)

Is Yawning Contagious?

- We're going to be doing hypothesis testing via **simulation**
- Intuitively:
“If we assume there is no effect, and we did a random trial, what is the probability of seeing a result like what we saw?”

✓
0s

```
[9] control_yawners = 4 / (12 + 4)  
    treat_yawners = 10 / (24 + 10)
```

```
control_yawners, treat_yawners
```



```
(0.25, 0.29411764705882354)
```



```
total_yawners = (4 + 10) / (12 + 4 + 24 + 10)  
total_yawners
```



```
0.28
```

Hypothesis Testing

```
▶ test_stat = treat_yawners - control_yawners  
test_stat
```

```
⇒ 0.04411764705882354
```

In our real data, we saw an 0.04 difference, but in the null world, we saw a 0.007 difference. Much smaller!

... but just one trial

```
people = ("no yawn" * 12 + "yawn" * 4  
          + "no yawn" * 24 + "yawn" * 10)  
random.shuffle(people)  
c_room = people[0:16]  
t_room = people[16:]  
c_yawners = sum(x == "yawn" for x in c_room) / len(c_room)  
t_yawners = sum(x == "yawn" for x in t_room) / len(t_room)  
null_world_test_stat = t_yawners - c_yawners  
null_world_test_stat
```

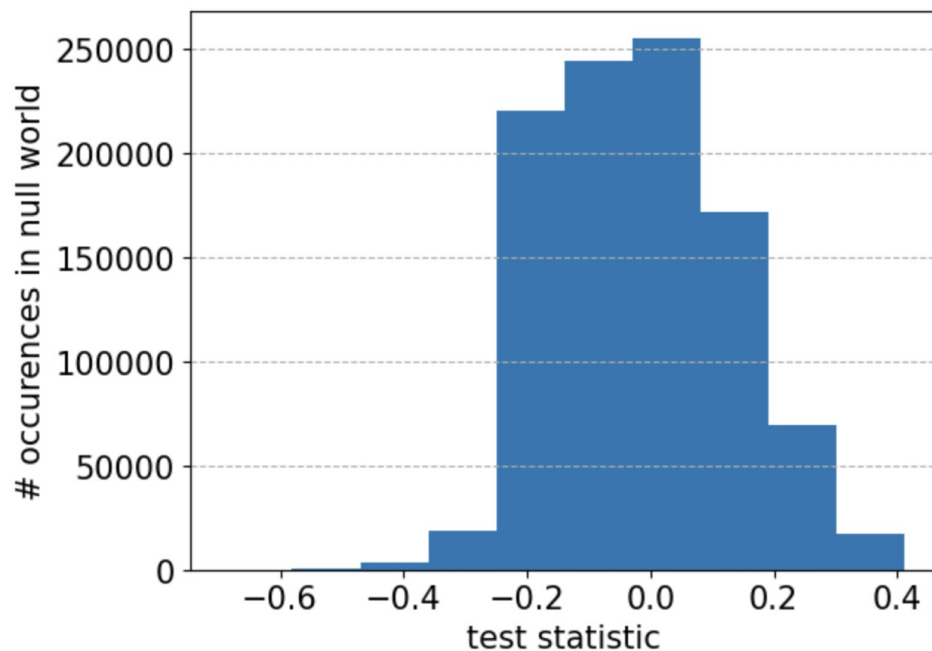
```
0.22794117647058826
```

Hypothesis Testing

```
import random
random.seed(42)
```

```
all_test_stats = []
for _ in range(1_000_000):
    people = (["no yawn"] * 12 + ["yawn"] * 4
              + ["no yawn"] * 24 + ["yawn"] * 4)
    random.shuffle(people)
    c_room = people[0:16]
    t_room = people[16:]
    c_yawners = sum(x == "yawn" for x in c_room)
    t_yawners = sum(x == "yawn" for x in t_room)
    null_world_test_stat = t_yawners - c_yawners
    all_test_stats.append(null_world_test_stat)
```

```
plt.hist(all_test_stats, bins=10)
plt.xlabel("test statistic")
plt.ylabel("# occurrences in null world")
plt.grid(axis="y", ls="--")
```

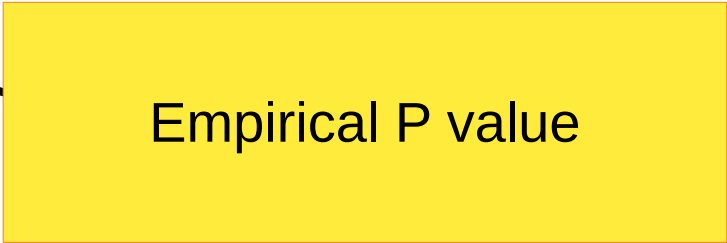


Hypothesis Testing

- How often was our simulated test statistic in the null world equal or stronger than what we observed?

```
[41] sum(x >= test_stat for x in all_ts) / len(all_ts)
```

⇒ 0.397975



Empirical P value

Hypothesis Testing

- P values can be confusing.

Assuming the null hypothesis (treat & control are the same), there is a $p=39.8\%$ chance of observing an equal or stronger effect

There is only a 39.8% chance the null hypothesis is true!

FALSE

Assuming null, If you did this experiment 1000 times, you would expect 398 runs to show an equal or stronger effect

There is a $100 - p = 60\%$ chance that yawning is contagious!

FALSE

Vectorization

- Python, as a programming language, often trades speed for ease-of-use.
- ... but we've seen that doing thing with Python libraries can be fast!
- DuckDB, Polars
- These libraries do all the hard work for you, and give you a nice interface
- Another similar tool: NumPy (numeric python)



Vectorize

```
%%time
rng = np.random.default_rng(seed=42)
masses = np.array(chinstraps["body_mass_g"]
                  + gentoos["body_mass_g"]
```

```
indexes = np.arange(0, len(all_penguins)
indexes = np.tile(indexes, (100_000, 1)
indexes = rng.permuted(indexes, axis=1)
samples = masses[indexes]
c_means = np.mean(samples[:,0:len(chinstraps)], axis=1)
g_means = np.mean(samples[:,len(chinstraps):], axis=1)
all_test_stats = g_means - c_means
```

```
CPU times: user 475 ms, sys: 73.2 ms, total: 548 ms
Wall time: 800 ms
```

```
%%time
all_penguins = (chinstraps["body_mass_g"].to_list()
               + gentoos["body_mass_g"].to_list())

all_test_stats = []
for _ in range(100_000):
    random.shuffle(all_penguins)
    null_chinstraps = all_penguins[:len(chinstraps)]
    null_gentoos = all_penguins[len(chinstraps):]
    diff = np.mean(null_gentoos) - np.mean(null_chinstraps)
    all_test_stats.append(diff)
```

```
CPU times: user 6.88 s, sys: 10.5 ms, total: 6.89 s
Wall time: 8.1 s
```

Big improvement!

Outline

- Course logistics
- Brief review
- **Linear models: interpretation**
- Linear models: analysis

```
!wget -nc https://rmarcus.info/data/health.csv
df = pd.read_csv("health.csv")
df.info()
```

```
>>> <class 'pandas.core.frame.DataFrame'>
RangeIndex: 2864 entries, 0 to 2863
Data columns (total 21 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Country                              2864 non-null   object
1   Region                               2864 non-null   object
2   Year                                  2864 non-null   int64
3   Infant_deaths                        2864 non-null   float64
4   Under_five_deaths                   2864 non-null   float64
5   Adult_mortality                     2864 non-null   float64
6   Alcohol_consumption                 2864 non-null   float64
7   Hepatitis_B                         2864 non-null   int64
8   Measles                             2864 non-null   int64
9   BMI                                  2864 non-null   float64
10  Polio                               2864 non-null   int64
11  Diphtheria                          2864 non-null   int64
12  Incidents_HIV                       2864 non-null   float64
13  GDP_per_capita                      2864 non-null   int64
14  Population_mln                      2864 non-null   float64
15  Thinness_ten_nineteen_years         2864 non-null   float64
16  Thinness_five_nine_years            2864 non-null   float64
17  Schooling                           2864 non-null   float64
18  Economy_status_Developed            2864 non-null   object
19  Economy_status_Developing           2864 non-null   object
20  Life_expectancy                     2864 non-null   float64

dtypes: float64(11), int64(8), object(2)
memory usage: 470.0+ KB
```

models

to get

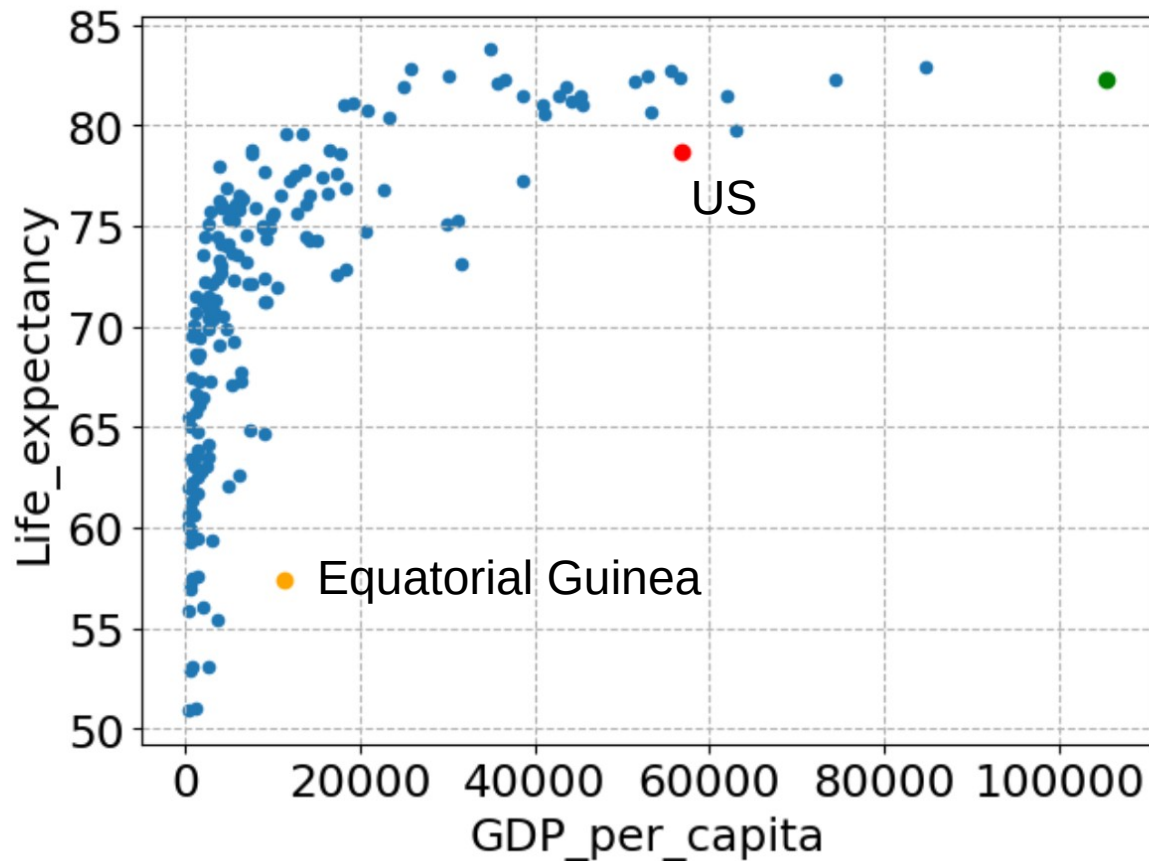
isn't binary?

How does GDP per capita relate to life expectancy?

```
✓ [55] y2015 = df[df["Year"] == 2015]
```

```
fig, ax = plt.subplots(constrained_layout=True)
y2015.plot.scatter("GDP_per_capita", "Life_expectancy", ax=ax)
ax.grid(ls="--")
ax.scatter([56763], [78.7], c="red")
ax.scatter([105462], [82.3], c="green")
ax.scatter([11283], [57.4], c="orange")
```

<matplotlib.collections.PathCollection at 0x7db5e73538b0>



Luxembourg

Seems like a clear
relationship – can we
model it?

$$\text{LifeExp} = a * \text{GDPpc} + b$$

$$\text{LifeExp} = a * \text{GDPpc} + b$$

How to find a and b ?

```
[59] from sklearn.linear_model import LinearRegression  
lr = LinearRegression()  
lr.fit(y2015[["GDP_per_capita"]], y2015["Life_expectancy"])  
lr.coef_, lr.intercept_
```

⇒ (array([0.00027818]), 67.95379283856532)

On average, when GDPpc increases by 1, life expectancy increase by 0.00028 years

a , the slope

b , the intercept

Linear Models

- How do we know these are the “right” parameters?
- The squared error loss function is convex
- assuming your input features are linearly independent!

$$\text{LifeExp} = a * \text{GDPpc} + b$$



⋮ ▾ Part 2: Descriptive & Predictive Modeling

⋮  09 - Linear Models.pdf

⋮  (Video) How is linear regression computed? 

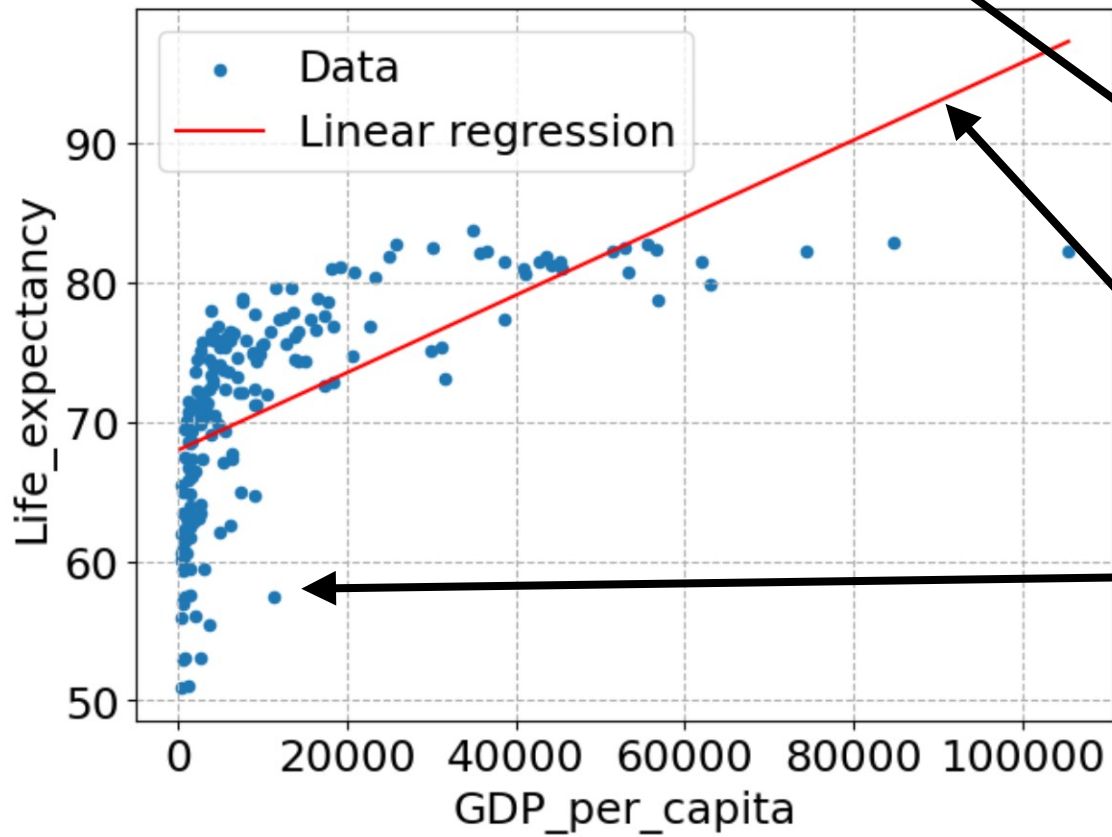
```
[11] fig, ax = plt.subplots(constrained_layout=True)
y2015.plot.scatter("GDP_per_capita", "Life_expectancy", ax=ax, label="Data")
ax.grid(ls="--")

x = np.linspace(y2015["GDP_per_capita"].min(), y2015["GDP_per_capita"].max(), 10)
pred_y = lr.coef_[0] * x + lr.intercept_
ax.plot(x, pred_y, color="red", label="Linear regression")
ax.legend()
```

Use numpy to apply our prediction to test input data

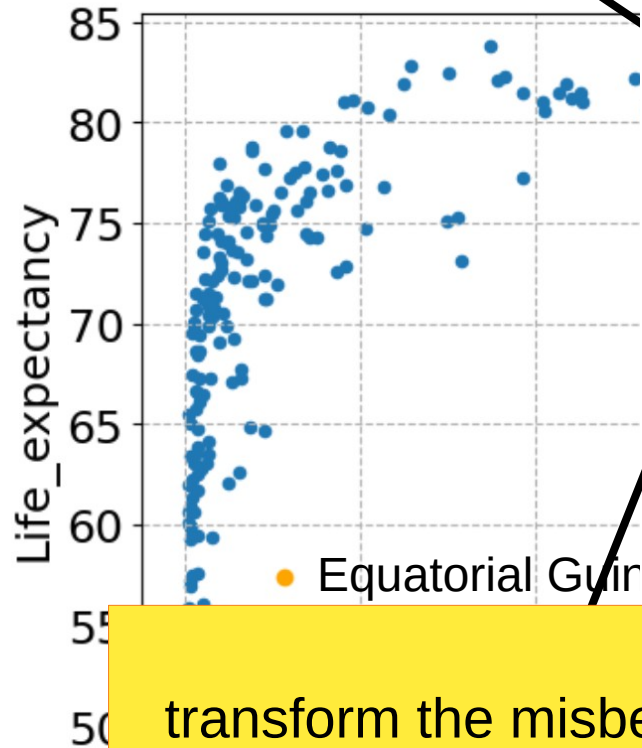
Plot the model's predictions

Not good!

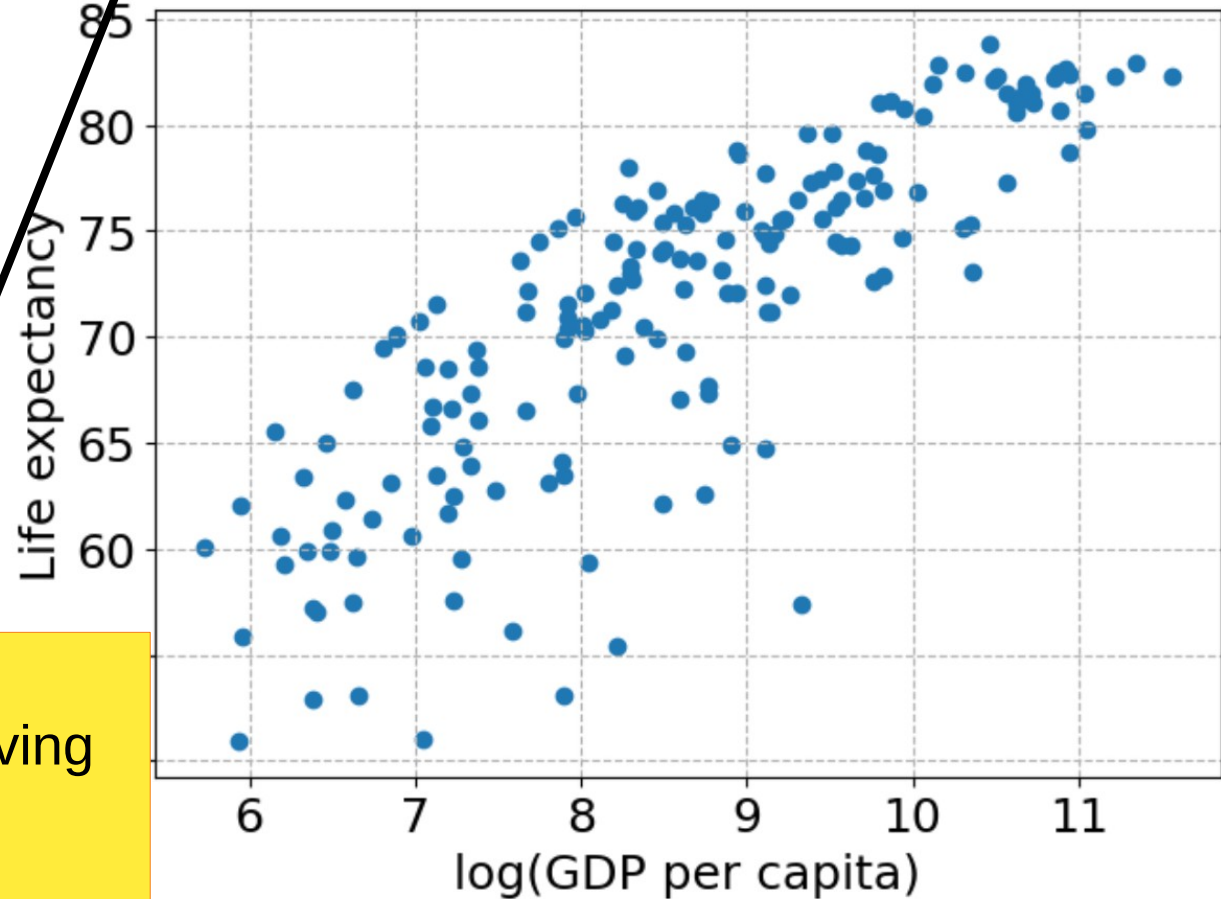


Not really a linear relationship, so our conclusion is suspect

```
lgdpc = np.log(y2015["GDP_per_capita"].to_numpy())  
fig, ax = plt.subplots(constrained_layout=True)  
ax.scatter(lgdpc, y2015["Life_expectancy"])  
ax.set_ylabel("Life expectancy")  
ax.set_xlabel("log(GDP per capita)")  
plt.grid(ls="--")
```



transform the misbehaving dimension



```
[59] from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(y2015[["GDP_per_capita"]], y2015["Life_expectancy"])
lr.coef_, lr.intercept_
```

⇒ (array([0.00027818]), 67.95379283856532)

$$\text{LifeEx} = 0.00027818 * \text{GDPpc} + 67.9537$$

```
[74] from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(lgdpc.reshape(-1, 1), y2015["Life_expectancy"])
lr.coef_, lr.intercept_
```

⇒ (array([4.62409602]), 31.81983460335335)

$$\text{LifeEx} = 4.62409602 * \log(\text{GDPpc}) + 31.81983$$

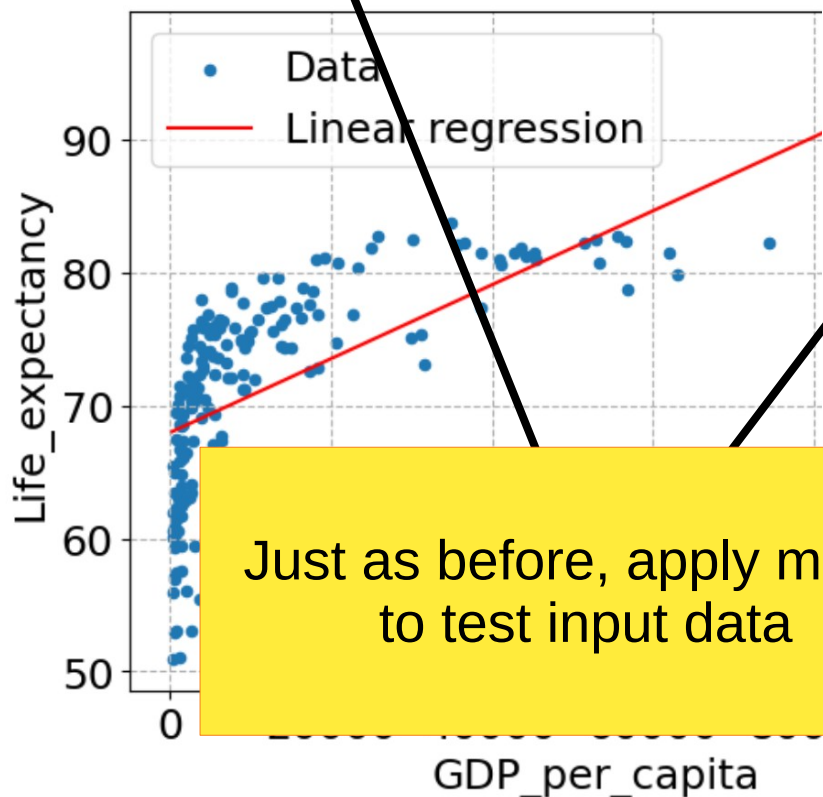
```
lr.fit(y2015[["GDP_per_capita"]], y2015["Life_expectancy"])
fig, ax = plt.subplots(constrained_layout=True)
y2015.plot.scatter("GDP_per_capita", "Life_expectancy", ax=
ax.grid(ls="--")
```

```
x = np.linspace(y2015["GDP_per_capita"].min(), y2015["GDP_p
pred_y = lr.coef_[0] * x + lr.intercept_
ax.plot(x, pred_y, color="red", label="Linear regression")
ax.legend()
```

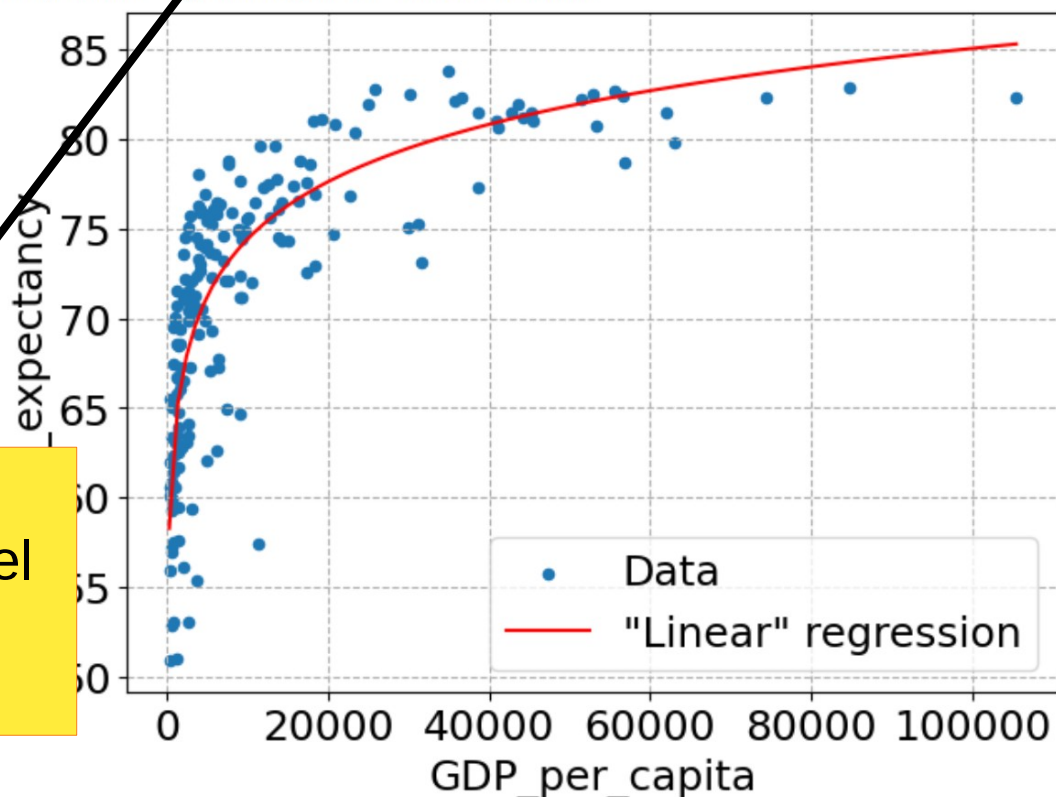
```
lr.fit(lgdpc.reshape(-1, 1), y2015["Life_expectancy"])
fig, ax = plt.subplots(constrained_layout=True)
y2015.plot.scatter("GDP_per_capita", "Life_expectancy", ax=ax, label="Data")
ax.grid(ls="--")
```

```
x = np.linspace(y2015["GDP_per_capita"].min(), y2015["GDP_per_capita"].max(), 100)
pred_y = lr.coef_[0] * np.log(x) + lr.intercept_
ax.plot(x, pred_y, color="red", label="\\"Linear\\" regression")
ax.legend()
```

<matplotlib.legend.Legend at 0x7db5e6884b20>



<matplotlib.legend.Legend at 0x7db5e6991a20>



```
[74] from sklearn.linear_model import LinearRegression
      lr = LinearRegression()
      lr.fit(lgdpc.reshape(-1, 1), y2015["Life_expectancy"])
      lr.coef_, lr.intercept_
```

⇒ (array([4.62409602]), 31.81983460335335)

$$\text{LifeEx} = 4.62409602 * \log(\text{GDPpc}) + 31.81983$$

What happens if **GDPpc** goes up by 1? Hard to say.
... but what if **GDPpc** doubles?

$$\text{NewLifeEx} = 4.624 * \log(2 * \text{GDPpc}) + 31.81983$$

$$\text{NewLifeEx} = 4.624 * [\log(2) + \log(\text{GDPpc})] + 31.81983$$

$$\text{NewLifeEx} = [4.624 \log(2)] + [4.624 \log(\text{GDPpc}) + 31.81983]$$

$$\text{NewLifeEx} = [4.624 \log(2)] + \text{LifeEx}$$

$$\text{NewLifeEx} = 3.21 + \text{LifeEx}$$

```
[74] from sklearn.linear_model import LinearRegression
      lr = LinearRegression()
      lr.fit(lgdpc.reshape(-1, 1), y2015["Life_expectancy"])
      lr.coef_, lr.intercept_
```

➡ (array([4.62409602]), 31.81983460335335)

$$\text{LifeEx} = 4.62409602 * \log(\text{GDPpc}) + 31.81983$$

What happens if **GDPpc** goes up by 1? Hard to say.
... but what if **GDPpc** doubles?

$$\text{NewLifeEx} = 4.624 * \log(2 * \text{GDPpc}) + 31.81983$$

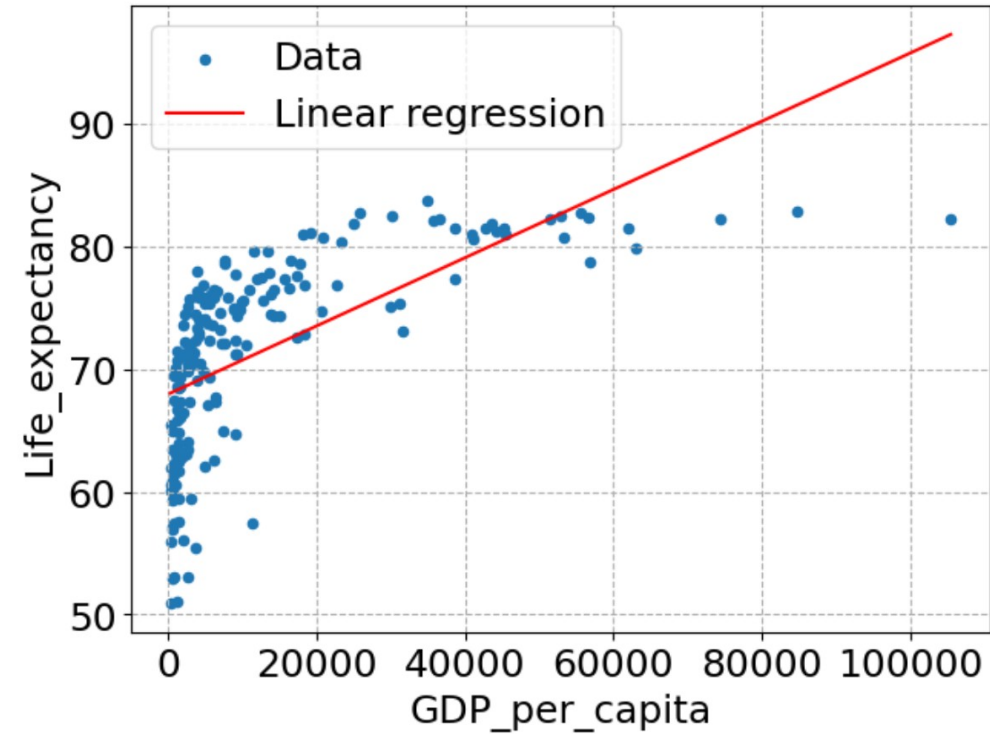
$$\text{NewLifeEx} = 4.624 * [\log(2) + \log(\text{GDPpc})] + 31.81983$$

$$\text{NewLifeEx} = [4.624 \log(2)] + [4.624 * \log(\text{GDPpc})] + 31.81983$$

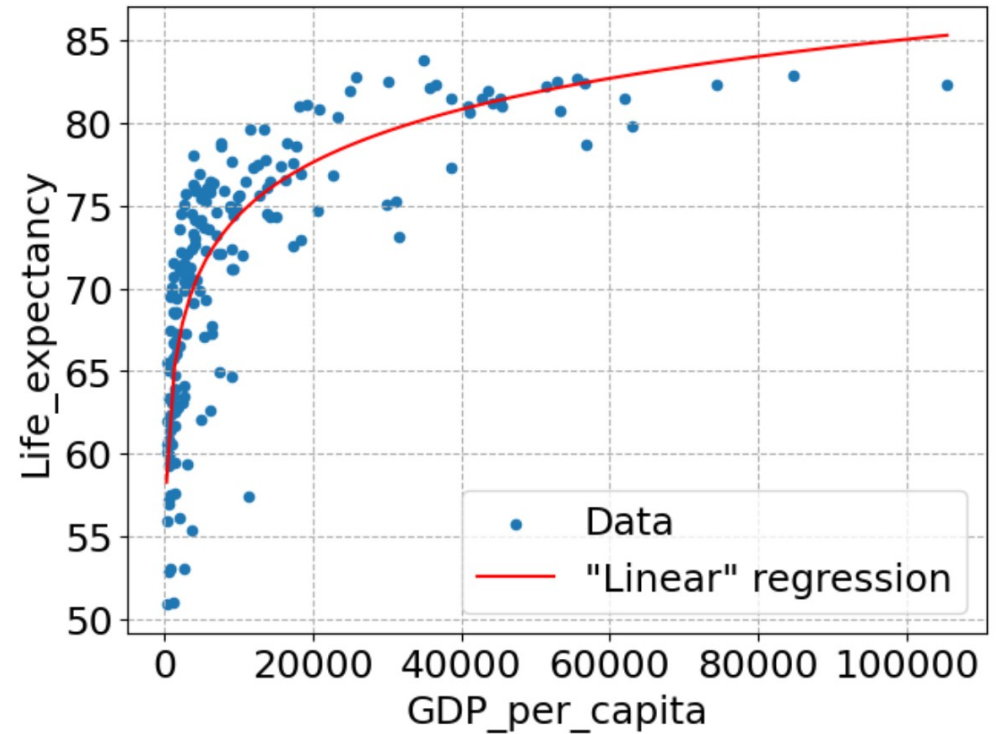
$$\text{NewLifeEx} = [4.624 \log(2)] + \text{LifeEx}$$

$$\text{NewLifeEx} = 3.21 + \text{LifeEx}$$

On average, If you double GDPpc, life expectancy goes up by 3.21 years

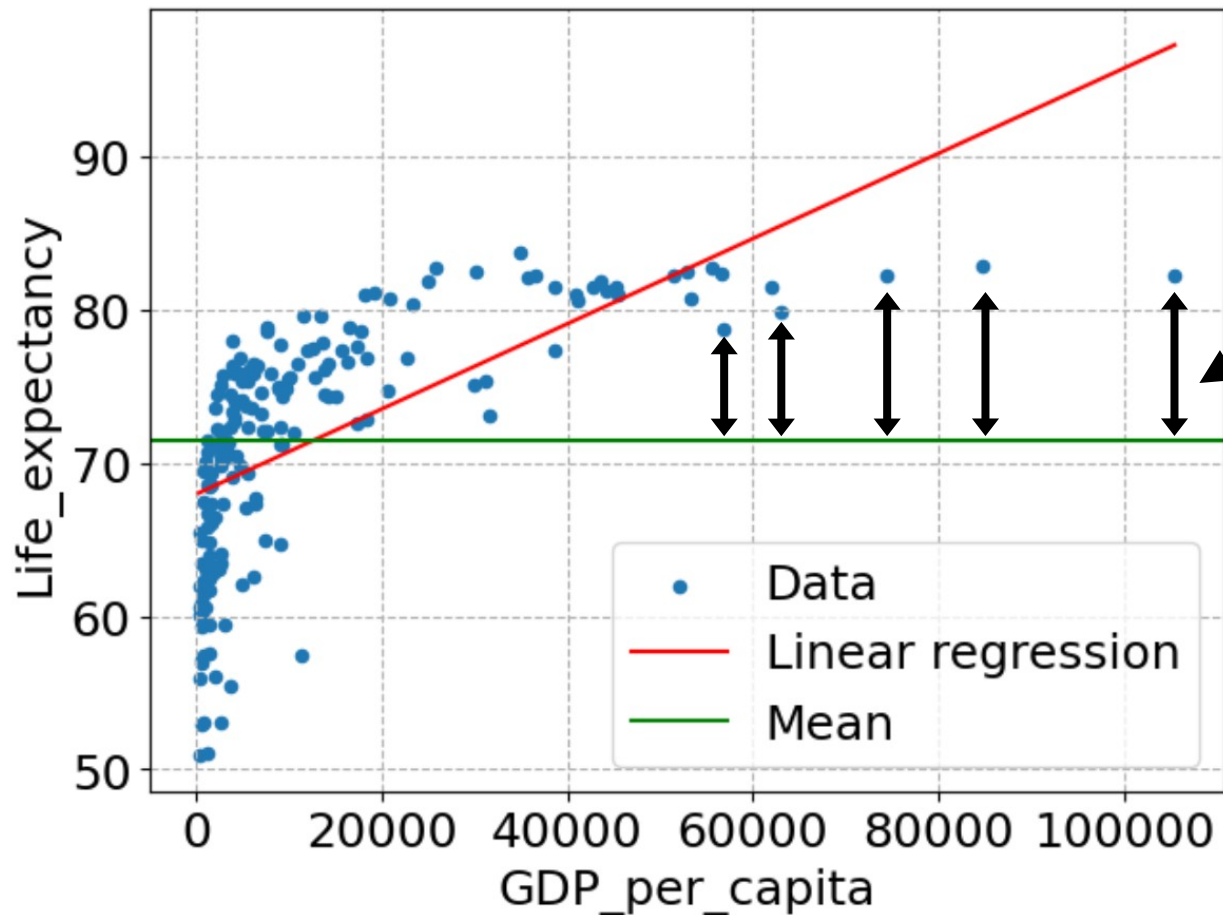


On average, when GDPpc increases by 1, life expectancy increase by 0.00028 years



On average, If you double GDPpc, life expectancy goes up by 3.21 years

Variance

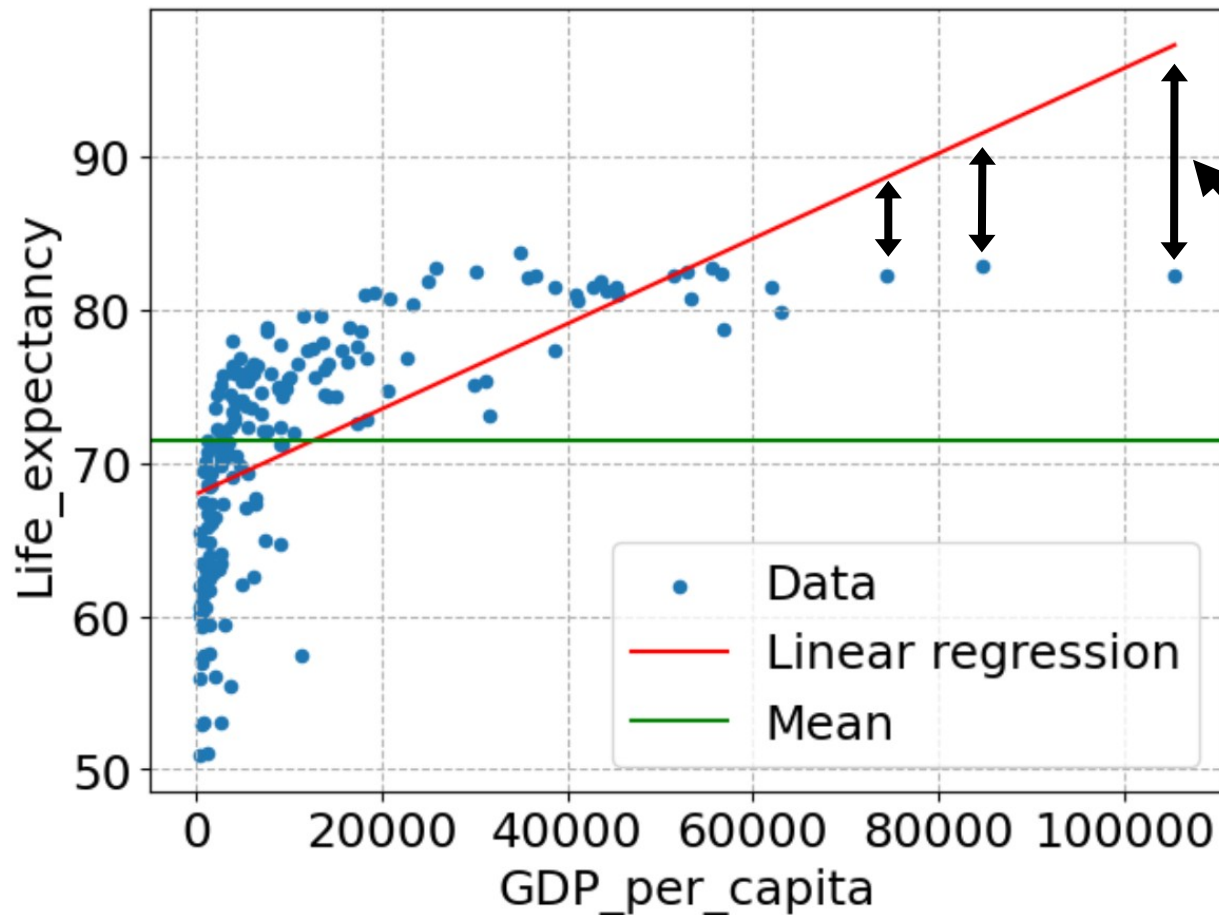


Variance of the data:

$$\text{Var}(Y) = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y})^2$$

Average squared distance
to mean

Variance



Variance of the data:

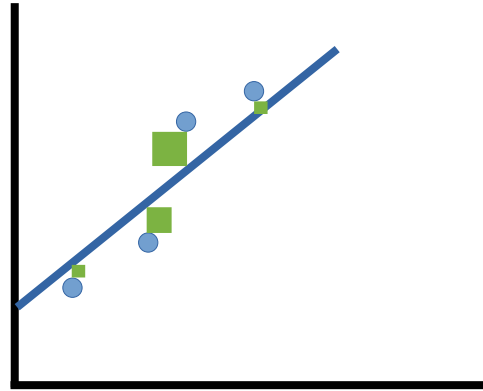
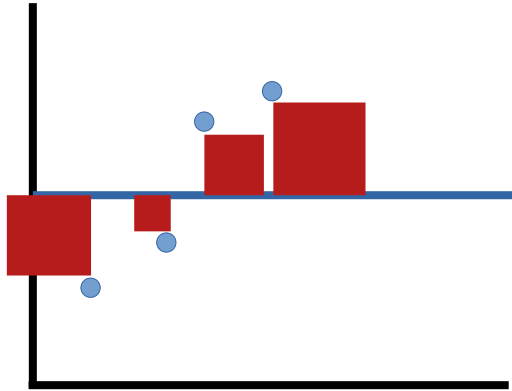
$$\text{Var}(Y) = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y})^2$$

Average squared distance
to prediction

“Variance” of the model (MSE):

$$\text{Var}_M(Y) = \frac{1}{n} \sum_{i=1}^n (Y_i - M(X_i))^2$$

Explained Variance



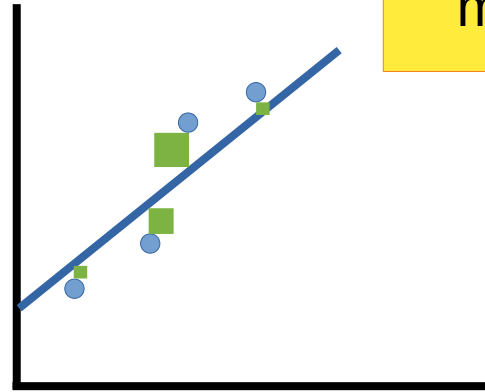
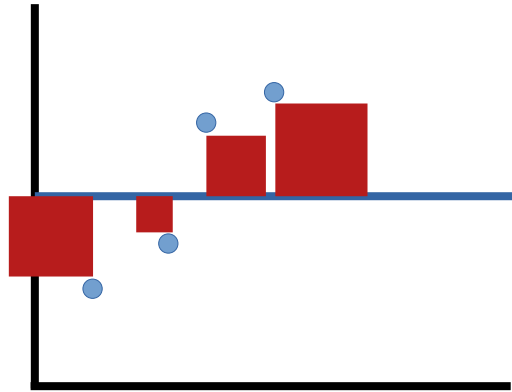
model variance



total variance



Explained Variance



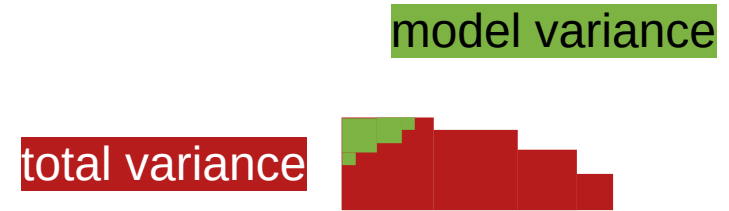
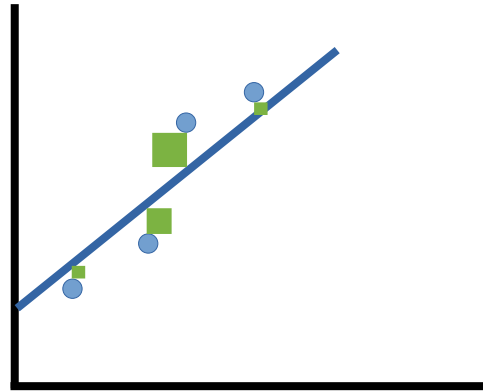
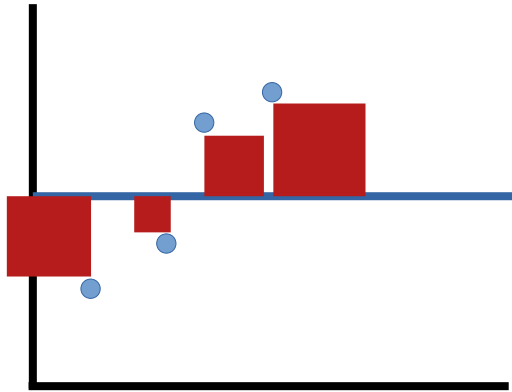
Variance after
modeling (green)

model variance

total variance

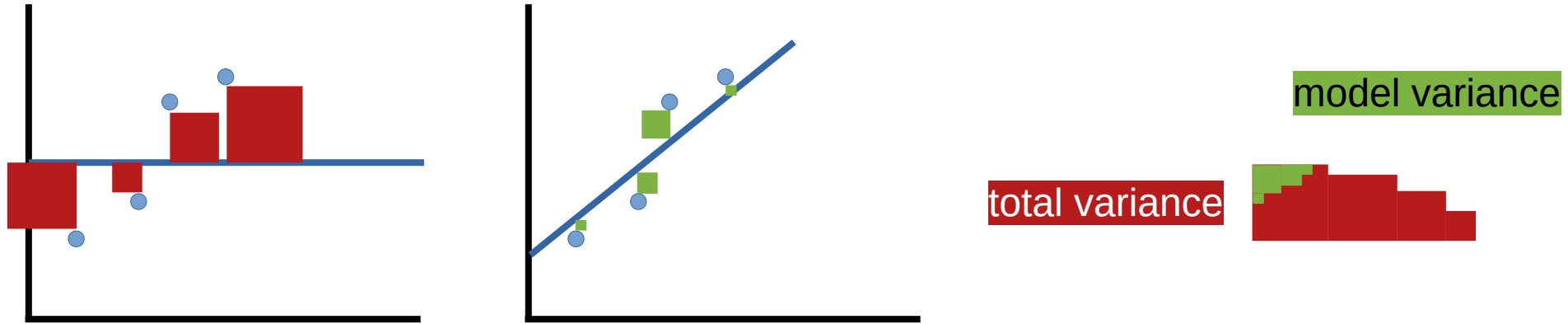
Variance before
modeling (red)

Explained Variance



$$R^2 = 1 - \frac{\text{variance after modeling}}{\text{variance before modeling}}$$

Explained Variance



$$R^2 = 1 - \frac{\text{variance after modeling}}{\text{variance before modeling}} = 1 - \frac{\frac{1}{n} \sum_{i=0}^n (Y_i - M(X_i))^2}{\frac{1}{n} \sum_{i=0}^n (Y_i - \hat{Y})^2} = 1 - \frac{\sum_{i=0}^n (Y_i - M(X_i))^2}{\sum_{i=0}^n (Y_i - \hat{Y})^2}$$

Close to 1 → small model variance compared to big total, good!

Close to 0 → nearly identical model and data variance, bad!

✓
0s [97] from sklearn.metrics import r2_score

✓
0s [102] lr = LinearRegression()
lr.fit(y2015[["GDP_per_capita"]], y2015["Life_expectancy"])
predicted = lr.predict(y2015[["GDP_per_capita"]])
r2_score(y2015["Life_expectancy"], predicted)

⇒ 0.39608518603719944

✓
0s [103] log_lr = LinearRegression()
log_lr.fit(lgdpc.reshape(-1, 1), y2015["Life_expectancy"])
predicted = log_lr.predict(lgdpc.reshape(-1, 1))
r2_score(y2015["Life_expectancy"], predicted)

⇒ 0.6726782300293612

Train models and get
predicted values

Compute R^2

✓
0s [97] `from sklearn.metrics import r2_score`

✓
0s [102] `lr = LinearRegression()
lr.fit(y2015[["GDP_per_capita"]], y2015["Life_expectancy"])
predicted = lr.predict(y2015[["GDP_per_capita"]])
r2_score(y2015["Life_expectancy"], predicted)`

⇒ 0.39608518603719944

→ 39% of the variance is explained by the linear relationship between GDPpc and life expectancy

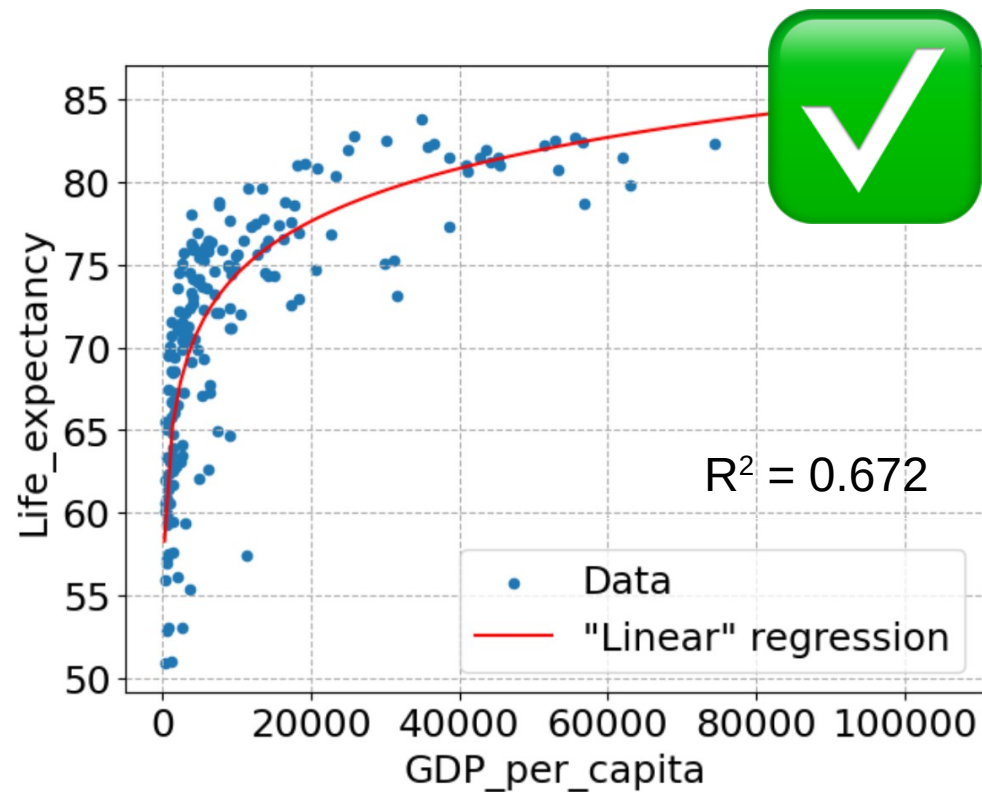
✓
0s [103] `log_lr = LinearRegression()
log_lr.fit(lgdpc.reshape(-1, 1), y2015["Life_expectancy"])
predicted = log_lr.predict(lgdpc.reshape(-1, 1))
r2_score(y2015["Life_expectancy"], predicted)`

⇒ 0.6726782300293612

→ 67% of the variance is explained by the linear relationship between *the log of* GDPpc and life expectancy



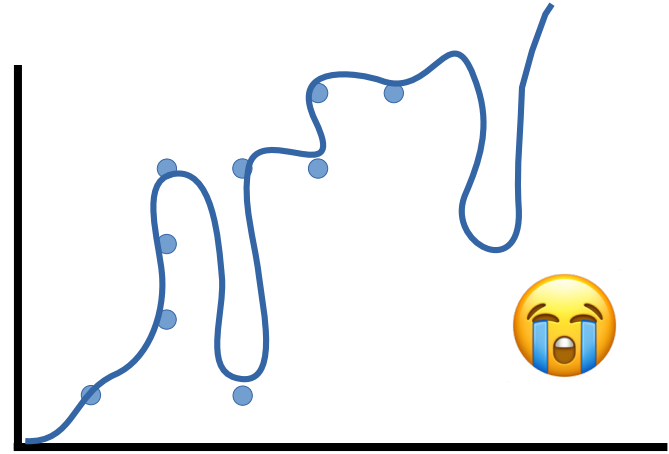
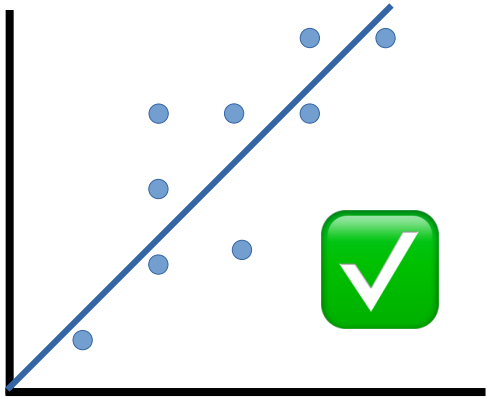
On average, when GDPpc increases by 1, life expectancy increase by 0.00028 years



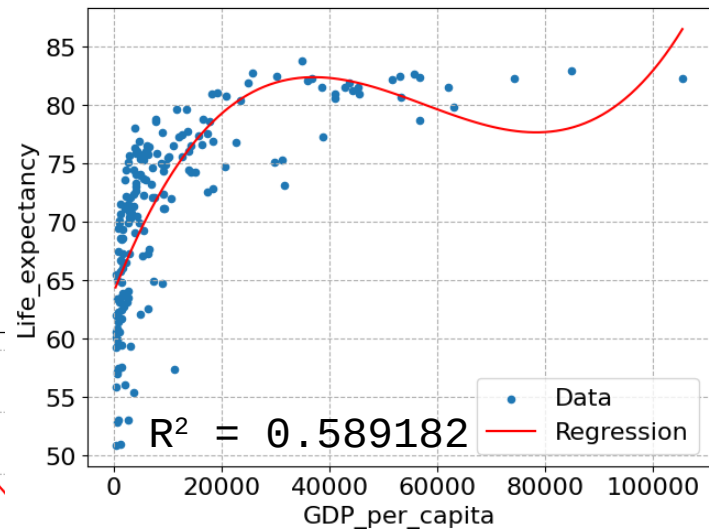
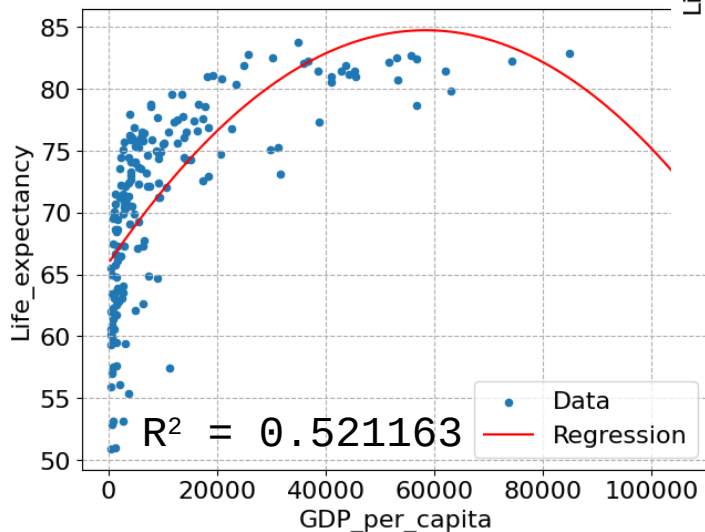
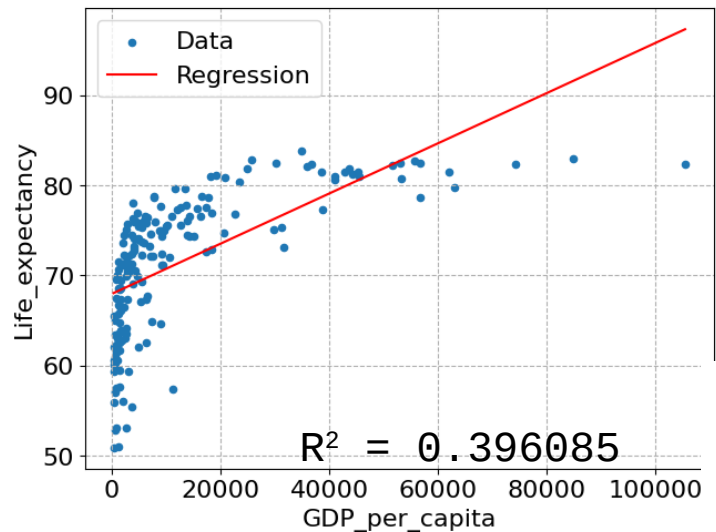
On average, If you double GDPpc, life expectancy goes up by 3.21 years

Overfitting

- Choosing a model based on fit only works when the model is *regularized* (constrained)



Overfitting



Outline

- Course logistics
- Brief review
- Linear models: interpretation
- **Linear models: analysis**

Significance?

- How do we know our results are significant?
- Hypothesis testing! Test stat = coefficient

```
[59] from sklearn.linear_model import LinearRegression  
     lr = LinearRegression()  
     lr.fit(y2015[["GDP_per_capita"]], y2015["Life_expectancy"])  
     lr.coef_, lr.intercept_
```

```
⇒ (array([0.00027818]), 67.95379283856532)
```

$$\text{LifeEx} = 0.00027818 * \text{GDPpc} + 67.9537$$

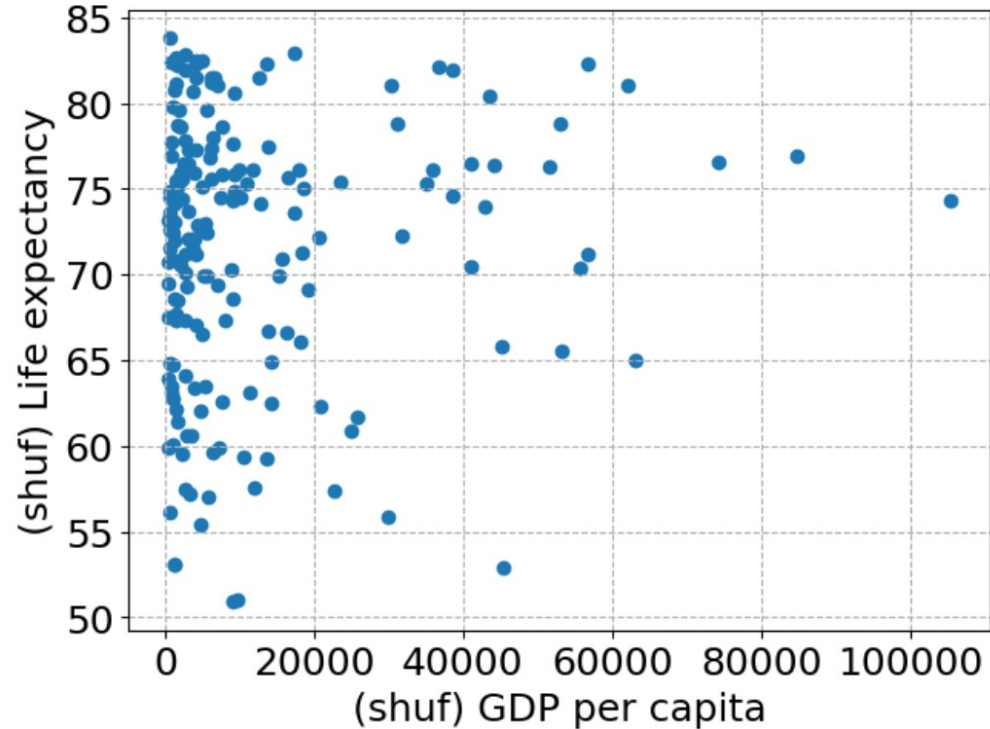
Make a copy of x
and shuffle it

~~permutat~~

- No
- Plot 1 trial from the null world
- the relationship between features and outputs is arbitrary.
- What is the distribution of the slope under this hypothesis?

```
# but is it significant? permutation test
# (null: we have all the data, but association is random)
x = y2015["GDP_per_capita"].to_numpy(copy=True)
y = y2015["Life_expectancy"].to_numpy()
np.random.shuffle(x)
```

```
fig, ax = plt.subplots(constrained_layout=True)
ax.scatter(x, y)
ax.set_xlabel("(shuf) GDP per capita")
ax.set_ylabel("(shuf) Life expectancy")
plt.grid(ls="--")
```



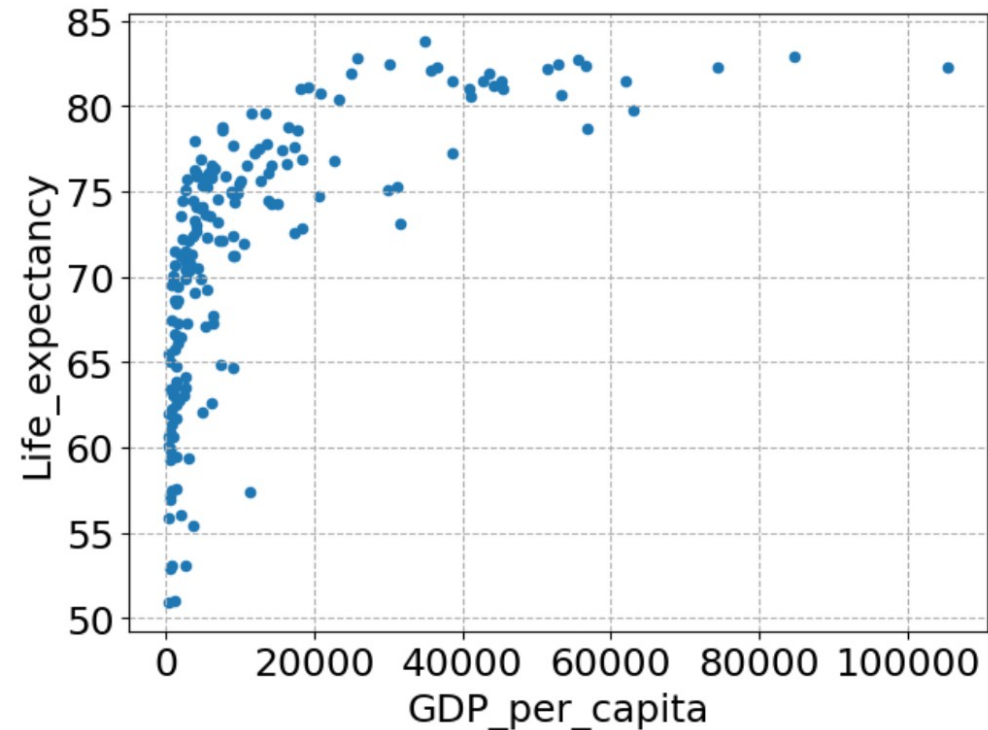
Make a copy of x
and shuffle it

~~permutat~~

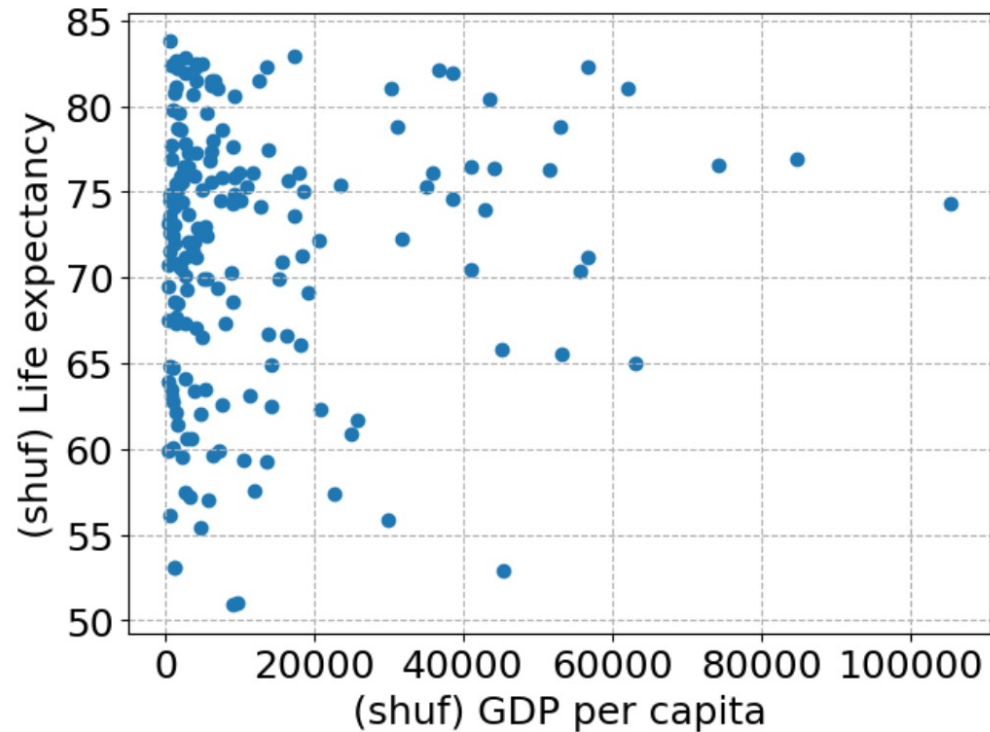
```
# but is it significant? permutation test  
# (null: we have all the data, but association is random)  
x = y2015["GDP_per_capita"].to_numpy(copy=True)  
y = y2015["Life_expectancy"].to_numpy()  
  
np.random.shuffle(x)
```

```
fig, ax = plt.subplots(constrained_layout=True)  
ax.scatter(x, y)  
ax.set_xlabel("(shuf) GDP per capita")  
ax.set_ylabel("(shuf) Life expectancy")  
plt.grid(ls="--")
```

- Plot 1 trial from the



⇒



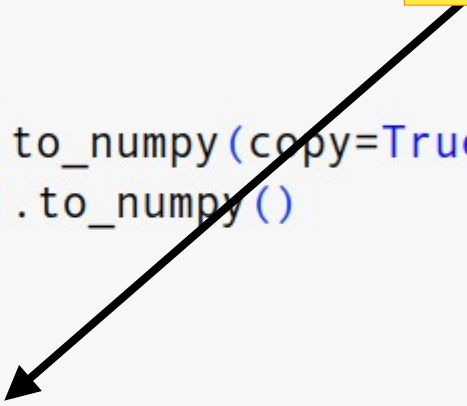
```
[15] all_coefs = []
```

```
for _ in range(20_000):  
    x = y2015["GDP_per_capita"].to_numpy(copy=True)  
    y = y2015["Life_expectancy"].to_numpy()  
    np.random.shuffle(x)  
  
    lr = LinearRegression()  
    lr.fit(x.reshape(-1, 1), y)  
    all_coefs.append(lr.coef_[0])
```

Do our simulation in a
loop



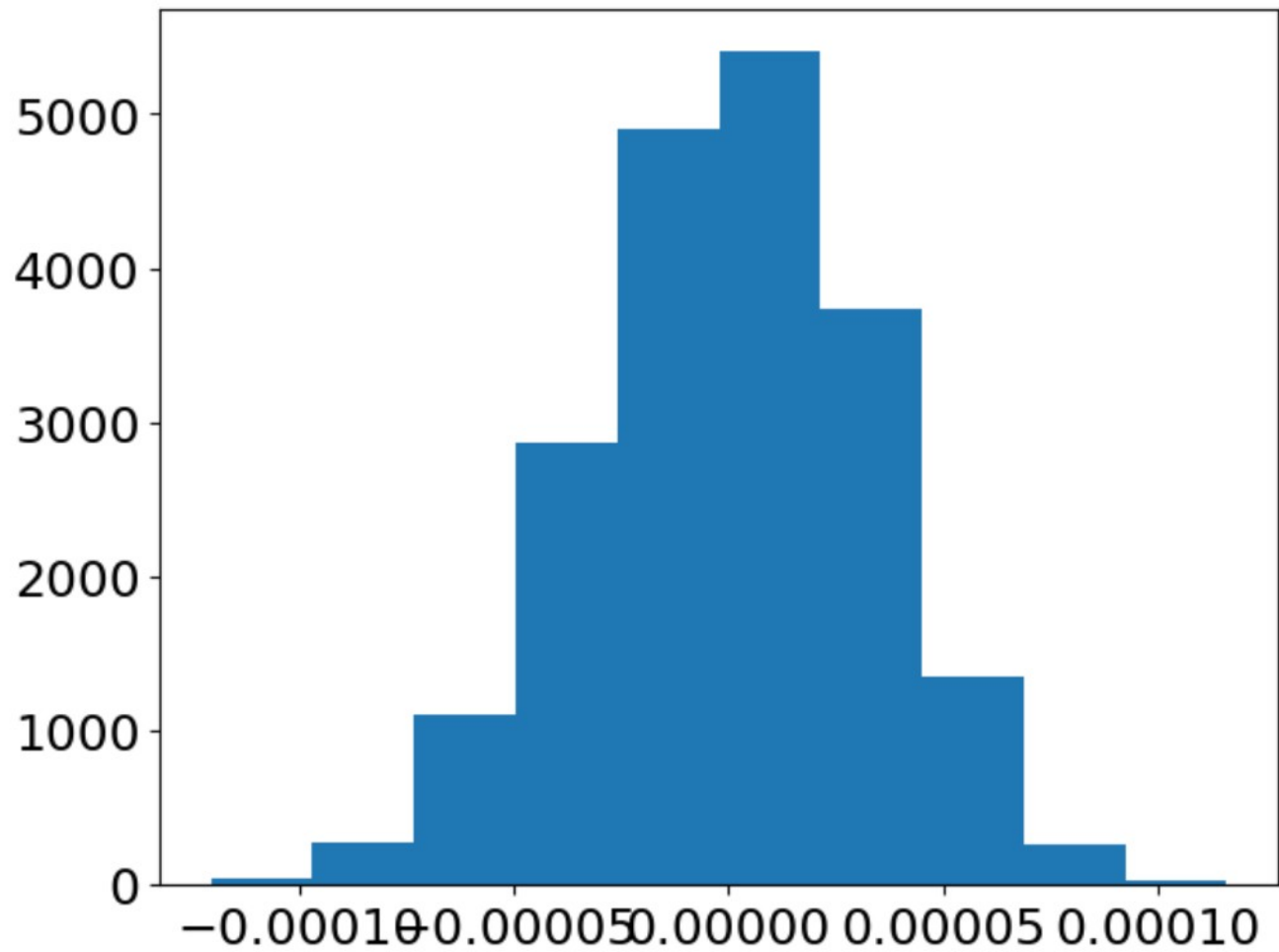
Measure the coefficient
under the null hypothesis



```
[15] all_coefs = []

    for _ in range(20000):
        x = y2015["GDP"]
        y = y2015["Lit"]
        np.random.shuffle(x)

        lr = LinearRegression()
        lr.fit(x.reshape(-1, 1), y)
        all_coefs.append(lr.coef_[0])
```



✓
0s



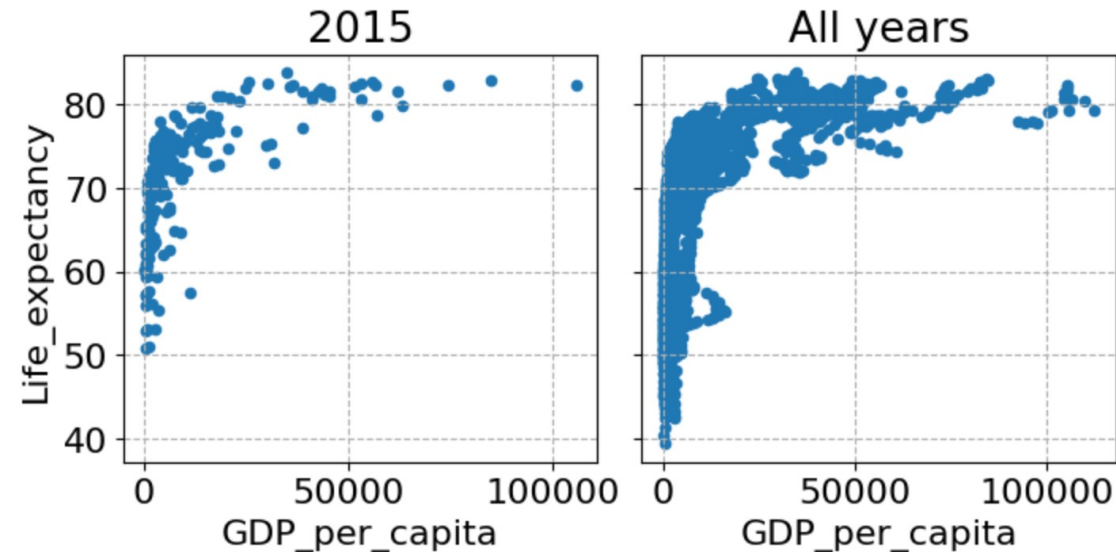
`sum(np.array(all_coefs) < 0.00027818)`



20000

Time is a Dimension

- We've been looking at just 2015 data, but we can look at more years.
 - Prior conclusions: if you go from a country with GDPpc of X to a country with GDPpc of X' *in the same year*...

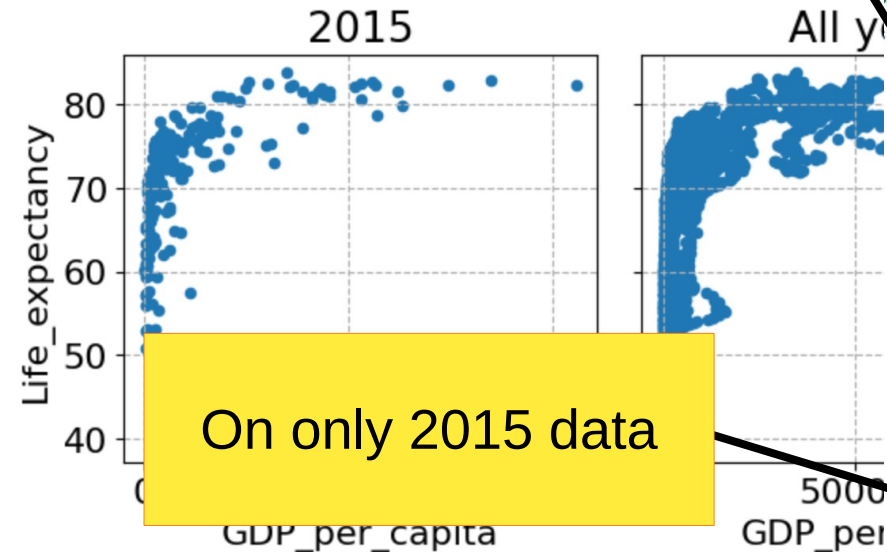


Time is a Dimension

- We've been looking at just 2015 data, but we can look at more years.
- Price you go from a country with GDPpc of X to a country with GDPpc of X'...

On all data

On only 2015 data



```
[16] from sklearn.linear_model import LinearRegression  
lr = LinearRegression()  
lr.fit(np.log(df[["GDP_per_capita"]]), df["Life_expectancy"])  
lr.coef_, lr.intercept_
```

```
(array([5.18114264]), 25.337803662610426)
```

```
from sklearn.linear_model import LinearRegression  
lr = LinearRegression()  
lr.fit(lgdpc.reshape(-1, 1), y2015["Life_expectancy"])  
lr.coef_, lr.intercept_
```

```
(array([4.62409602]), 31.81983460335335)
```

Average GPDpc and life expectancy over time

on Testi

- How can we test for significance over time?
- Possible null hypothesis: input/output pairing is arbitrary

Average is increasing year over year

What's wrong with assuming the pairing is arbitrary?

```
df.groupby("Year").agg(  
    P_GDP=("GDP_per_capita", "mean"),  
    P_life=("Life_expectancy", "mean")  
)
```

	P_GDP	P_life
Year		
2000	10247.614525	66.360335
2001	10343.301676	66.650279
2002	10462.452514	66.886592
2003	10661.150838	67.170391
2004	11040.413408	67.549721
2005	11301.547486	67.856425
2006	11708.994413	68.221788
2007	12084.128492	68.588268
2008	12126.184358	68.991061
2009	11622.201117	69.388827
2010	11840.396648	69.777095
2011	11983.329609	70.177095
2012	12093.687151	70.521788
2013	12176.139665	70.877095
2014	12345.960894	71.216760
2015	12617.296089	71.463687

Permutation Test

- **H**ypothesis testing
 - **P**ermutation test
- Aside: for folks who have learned about hypothesis testing before, this is *multicollinearity* of GDPpc and LifeEx on year
- ... how would you fix it? See if your answer is more or less complex than our fix...
- is arbitrary

```
df.groupby("Year").agg(  
    P_GDP=("GDP_per_capita", "mean"),  
    P_life=("Life_expectancy", "mean")  
)
```



P_GDP

P_life



Year

2000 10247.614525 66.360335

2001 10343.301676 66.650279

2002 10462.452514 66.886592

2003 10661.150838 67.170391

2004 11040.413408 67.549721

2005 11301.547486 67.856425

2006 11708.994413 68.221788

2007 12084.128492 68.588268

2008 12126.184358 68.991061

2009 11622.201117 69.388827

2010 11840.396648 69.777095

2011 11983.329609 70.177095

2012 12093.687151 70.521788

2013 12176.139665 70.877095

2014 12345.960894 71.216760

2015 12617.296089 71.463687

Permutation Testing

- How do we fix it?
- Null hypothesis:
input/output pairing is
arbitrary *within each*
year
- Preserves increased
averages over time in the
null world

```
shuffled = (df.groupby("Year")["GDP_per_capita", "Life_expectancy"]  
            .transform(np.random.permutation)  
            .reset_index(drop=True))
```

Group by year, apply a
random permutation

How do we fix it?

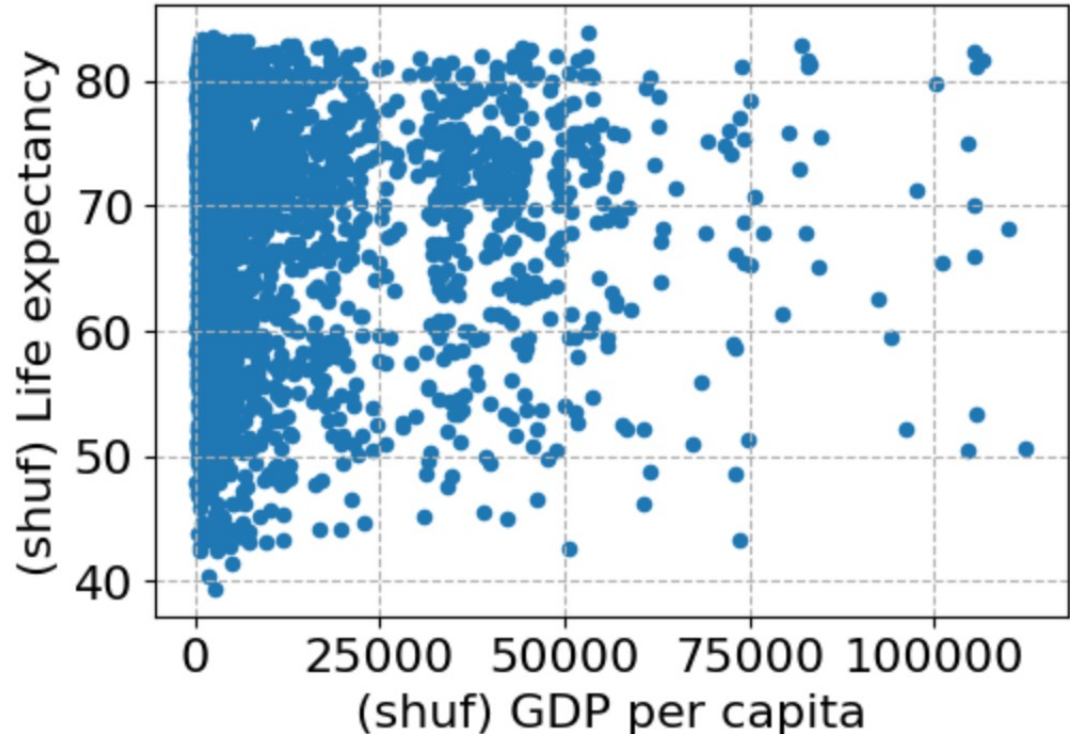
- Null hypothesis:
input/output pairing is
arbitrary *within each
year*
- Preserves increased
averages over time in the
null world

Group by year, apply a random permutation

```
shuffled = (df.groupby("Year")["GDP_per_capita", "Life_expectancy"]  
            .transform(np.random.permutation)  
            .reset_index(drop=True))
```

```
fig, ax = plt.subplots(constrained_layout=True, figsize=(5, 3.5))  
shuffled.plot.scatter("GDP_per_capita", "Life_expectancy", ax=ax)  
ax.set_xlabel("(shuf) GDP per capita")  
ax.set_ylabel("(shuf) Life expectancy")  
plt.grid(ls="--")
```

- Null hypothesis: input/output pair arbitrary *within* ϵ year
- Preserves increase averages over time null world



Put it in a loop

- How do we fix it?

- Collect test stats from null world

inputs can be arbitrary *within each year*

```
[41] all_coefs = []  
for _ in range(2000):  
    shuffled = (df.groupby("Year")["GDP_per_capita", "Life_expectancy"]  
                .transform(np.random.permutation)  
                .reset_index(drop=True))  
    lr = LinearRegression()  
    lr.fit(np.log(shuffled["GDP_per_capita"]), shuffled["Life_expectancy"])  
    all_coefs.append(lr.coef_[0])  
_ = plt.hist(all_coefs)
```

```
[16] from sklearn.linear_model import LinearRegression  
lr = LinearRegression()  
lr.fit(np.log(df["GDP_per_capita"]), df["Life_expectancy"])  
lr.coef_, lr.intercept_
```

```
↔ (array([5.18114164]), 25.337803662610426)
```

We don't see anything close to this strong



Interpretation

```
[16] from sklearn.linear_model import LinearRegression  
lr = LinearRegression()  
lr.fit(np.log(df[["GDP_per_capita"]]), df["Life_expectancy"])  
lr.coef_, lr.intercept_
```

All time

➡ (array([5.18114264]), 25.337803662610426) $5.181 * \log(2) = 3.59$

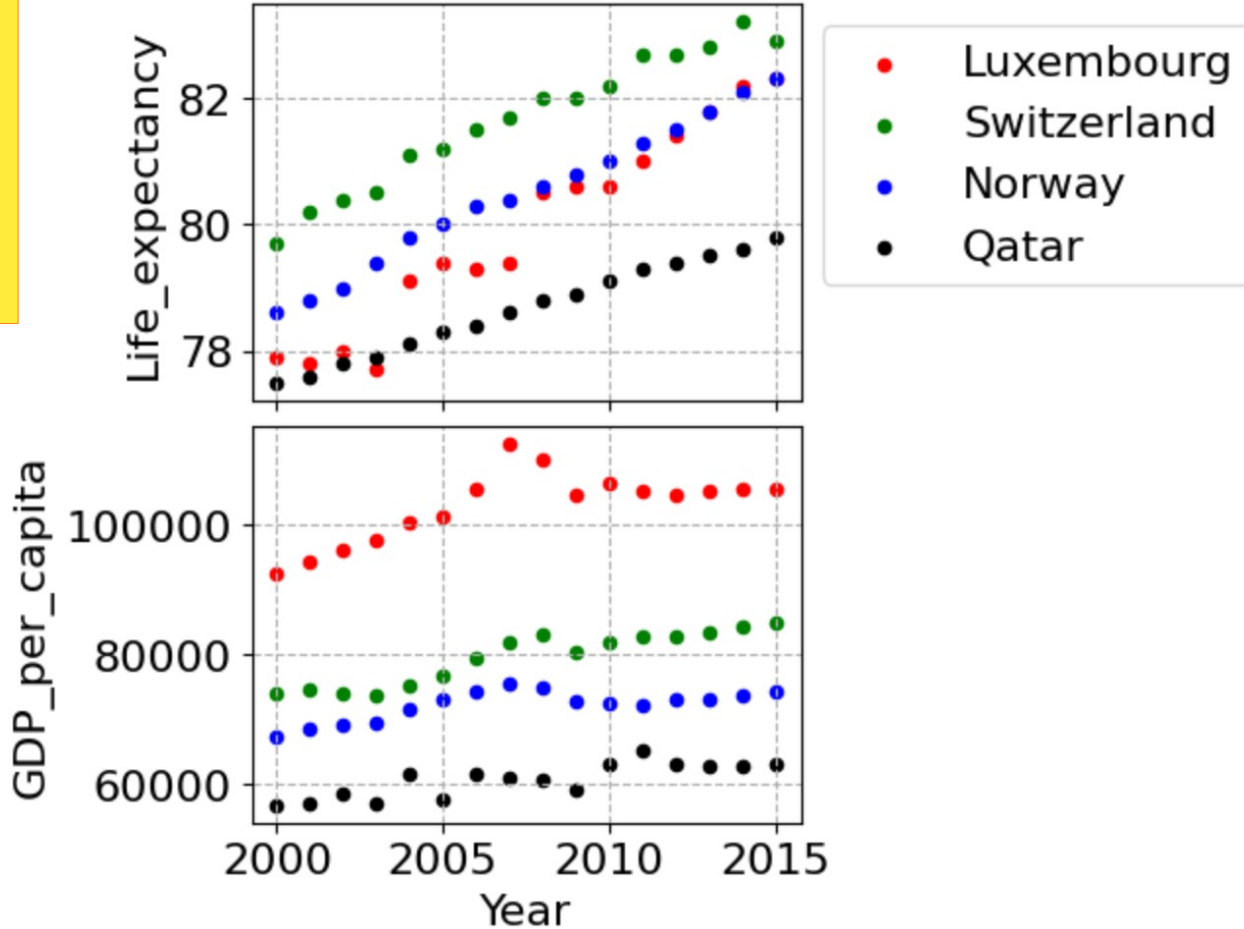
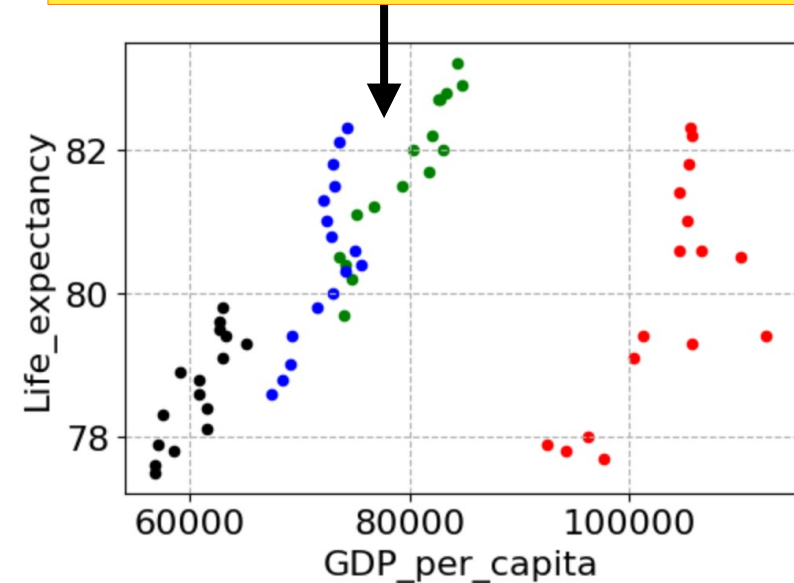
```
▶ from sklearn.linear_model import LinearRegression  
lr = LinearRegression()  
lr.fit(lgdpc.reshape(-1, 1), y2015["Life_expectancy"])  
lr.coef_, lr.intercept_
```

Only 2015

➡ (array([4.62409602]), 31.81983460335335) $4.624 * \log(2) = 3.21$

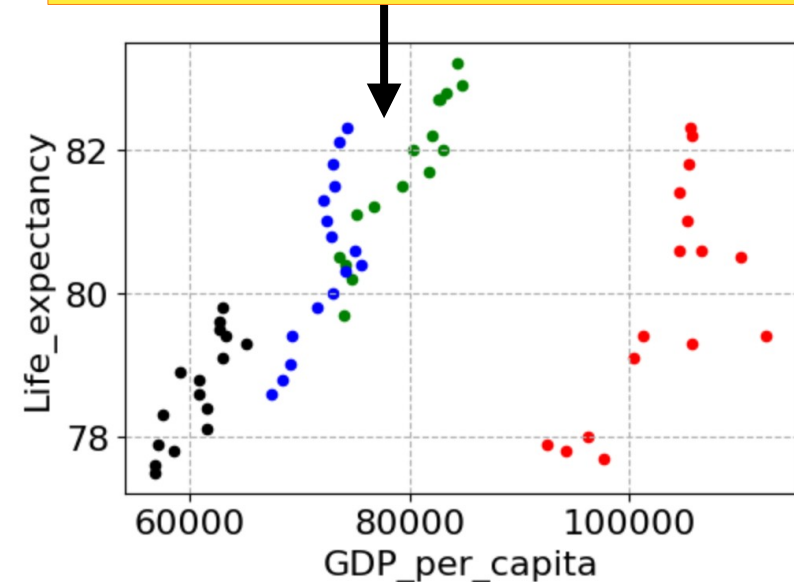
Grouped Samples

Steepness of slopes within groups can be viewed as the efficiency of converting GDPpc into LifeEx



Grouped Samples

Steepness of slopes within groups can be viewed as the efficiency of converting GDPpc into LifeEx



How would you design tests for:

- Is there a significant difference between two countries?
- Is the distribution of slopes significantly different than random?

Next Time

- What if I have more than one input / feature variable?